Pipeline Components Delivered:

1. Data Acquisition & Pipeline Design

• Spark SQL was used to generate 10,000 fictitious sales transactions

Other approaches were discussed:

• For columnar efficiency, parquet files from cloud storage (S3/Azure Data Lake)

• JDBC connections for database sources; streaming ingestion from Kafka/Event Hubs for real-time dashboards

• Rationale: SQL-based generation was preferred over single-machine pandas due to Databricks compatibility and scalability.

• Explanation: I used the sales department in an Ecommerce business as an example. The business offers products such as Electronics, Clothing, Home materials, Books and Sports equipment. For sample data I recorded sales from a day including fictional sales quantities and product prices along, with product ID order date transaction ID and customer information. id. One customer purchased 8 pieces of clothes at $163 so total is $1320, then, second customer purchased home materials 4 units @ $71.87 so total is $287.4, Similarly third and fourth customers purchased sport materials @$224.68 and $165 so total is $1797.4, and $496 respectively. Last customer purchased 4 electronic items @444.99 each so total is$1777.9.

```
# Import necessary libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import random

# Initialize Spark Session
spark = SparkSession.builder.appName("SalesDepartmentPipeline").getOrCreate()

print("✓ Spark Session Initialized")
print(f"Spark Version: {spark.version}")
```

```
SELECT
  concat('TXN', lpad(cast(id as string), 6, '0')) as transaction_id,
  concat('CUST', lpad(cast(cast(rand() * 1000 as int) as string), 4, '0')) as customer_id,
  date_add('2024-01-01', cast(rand() * 364 as int)) as order_date,
  concat('PROD', lpad(cast(cast(rand() * 50 as int) + 1 as string), 3, '0')) as product_id,
  CASE cast(rand() * 5 as int)
    WHEN 0 THEN 'Electronics'
    WHEN 1 THEN 'Clothing'
    WHEN 2 THEN 'Home'
    WHEN 3 THEN 'Books'
    ELSE 'Sports'
  END as category,
  cast(rand() * 9 as int) + 1 as quantity,
  cast(rand() * 490 + 10 as decimal(10,2)) as unit_price
FROM range(10000)
""")
```

```
Sample data:
+--------------+-----------+----------+----------+-----------+--------+----------+-----------
-+
|transaction_id|customer_id|order_date|product_id|   category|quantity|unit_price|total_amoun
t|
+--------------+-----------+----------+----------+-----------+--------+----------+-----------
-+
|     TXN000000|   CUST0019|2024-02-03|   PROD005|   Clothing|       8|    165.00|     1320.0
0|
|     TXN000001|   CUST0691|2024-02-15|   PROD021|       Home|       4|     71.87|      287.4
8|
|     TXN000002|   CUST0777|2024-08-30|   PROD023|     Sports|       8|    224.68|     1797.4
4|
|     TXN000003|   CUST0478|2024-03-07|   PROD018|     Sports|       3|    165.36|      496.0
8|
|     TXN000004|   CUST0609|2024-09-17|   PROD001|Electronics|       4|    444.99|     1779.9
6|
+--------------+-----------+----------+----------+-----------+--------+----------+-----------
-+
only showing top 5 rows
```

2. Data Transformation & Feature Engineering

• Calculated customer RFM metrics by Recency, Frequency, and Monetary value;
• added temporal features: year, month, quarter, day of week, week of year;
Developed computed columns such as Total Amount, Average Order Value, and Unique Products per Customer.

• Critical analysis: took a hybrid approach for maximum flexibility, comparing the DataFrame API with SQL-based approaches to problems.

```
# SECTION 2: DATA TRANSFORMATION & FEATURE ENGINEERING
# ====================================================

# Add date-based features for time-series analysis
df_sales_enriched = df_sales \
    .withColumn('year', year('order_date')) \
    .withColumn('month', month('order_date')) \
    .withColumn('quarter', quarter('order_date')) \
    .withColumn('day_of_week', dayofweek('order_date')) \
    .withColumn('week_of_year', weekofyear('order_date'))

# Customer-level aggregations for RFM analysis
from pyspark.sql.window import Window

customer_metrics = df_sales_enriched.groupBy('customer_id').agg(
    count('transaction_id').alias('frequency'),
    sum('total_amount').alias('monetary'),
    max('order_date').alias('last_purchase_date'),
    avg('total_amount').alias('avg_order_value'),
    countDistinct('product_id').alias('unique_products')
)

print("✓ Customer metrics calculated")
customer_metrics.show(10)
```

```
✓ Customer metrics calculated
+-----------+---------+--------+------------------+---------------+---------------+
|customer_id|frequency|monetary|last_purchase_date|avg_order_value|unique_products|
+-----------+---------+--------+------------------+---------------+---------------+
|   CUST0050|        5| 6829.26|        2024-12-25|    1365.852000|              5|
|   CUST0265|       11|14820.84|        2024-11-18|    1347.349091|             10|
|   CUST0584|        9|13890.28|        2024-10-22|    1543.364444|              9|
|   CUST0895|       12|16847.81|        2024-12-16|    1403.984167|             12|
|   CUST0259|       11|12120.57|        2024-12-29|    1101.870000|              9|
|   CUST0270|       12|16877.55|        2024-12-09|    1406.462500|             11|
|   CUST0623|        9|10689.98|        2024-12-25|    1187.775556|              9|
|   CUST0670|       14|15019.43|        2024-12-17|    1072.816429|             14|
|   CUST0582|       10|13210.85|        2024-12-25|    1321.085000|             10|
|   CUST0476|        9|17488.61|        2024-09-08|    1943.178889|              7|
+-----------+---------+--------+------------------+---------------+---------------+
```

Explanation: I calculated the metrics of customers with Total Amount, Last Purchase, Average Purchase Price, and Product Id. Here, a sample example of 10 customers is taken, showing details.

3. Spark DataFrames & SQL Queries

• DataFrame API: Type-safe customer metrics aggregations with groupBy() and agg()

Spark SQL: Business-friendly queries for top customers by lifetime value and revenue by category

• Methodological motivation: SQL for reporting accessible to analysts; DataFrame API for programmatic ML pipelines

```
print("✓ Tables registered for SQL access")

# Query 1: Revenue by Category
revenue_by_category = spark.sql("""
SELECT
    category,
    COUNT(*) as transaction_count,
    SUM(total_amount) as total_revenue,
    AVG(total_amount) as avg_transaction_value,
    SUM(quantity) as total_units_sold
FROM sales
GROUP BY category
ORDER BY total_revenue DESC
""")

print("\n=== Revenue by Category ===")
revenue_by_category.show()
```

```
# Query 2: Top 10 Customers by Revenue
top_customers = spark.sql("""
SELECT
    customer_id,
    frequency as purchase_count,
    ROUND(monetary, 2) as lifetime_value,
    ROUND(avg_order_value, 2) as avg_order,
    unique_products
FROM customer_stats
ORDER BY monetary DESC
LIMIT 10
""")

print("\n=== Top 10 Customers by Lifetime Value ===")
```

```
✓ Tables registered for SQL access

=== Revenue by Category ===
+-----------+-----------------+-------------+--------------------+----------------+
|   category|transaction_count|total_revenue|avg_transaction_value|total_units_sold|
+-----------+-----------------+-------------+--------------------+----------------+
|     Sports|             4136|   5315869.54|          1285.268264|           20715|
|Electronics|             1957|   2511042.50|          1283.108074|            9856|
|   Clothing|             1617|   2011488.24|          1243.963043|            7971|
|       Home|             1279|   1675144.12|          1309.729570|            6631|
|      Books|             1011|   1295933.73|          1281.833561|            5119|
+-----------+-----------------+-------------+--------------------+----------------+
```

```
=== Top 10 Customers by Lifetime Value ===
+-----------+--------------+--------------+---------+---------------+
|customer_id|purchase_count|lifetime_value|avg_order|unique_products|
+-----------+--------------+--------------+---------+---------------+
|   CUST0530|            20|      33839.42|  1691.97|             18|
|   CUST0119|            20|      33607.60|  1680.38|             19|
|   CUST0587|            21|      31728.09|  1510.86|             19|
|   CUST0364|            16|      30587.94|  1911.75|             12|
|   CUST0863|            15|      29795.54|  1986.37|             14|
|   CUST0142|            15|      29400.71|  1960.05|             11|
|   CUST0116|            15|      28432.34|  1895.49|             12|
|   CUST0365|            20|      28327.78|  1416.39|             15|
|   CUST0002|            16|      28023.48|  1751.47|             12|
|   CUST0551|            17|      27818.44|  1636.38|             14|
+-----------+--------------+--------------+---------+---------------+
```

Explanation: In table 1, it is shown which material is mostly sold on descending order from Sports materials to Books and its average transaction value and total no. of units sold.

However, on table 2, it shows frequent customer with most no. of products purchased, total no. of purchase cost in history, average cost of each order, and no. of products ordered.

4. Advanced Analytics

• Product category revenue analysis by unit sales and count of transactions

• Customer segment metrics highlighting frequency, monetary value, and product variability

• The top 10 customers, by lifetime value, by purchase frequency

```
# Analyze segments
segment_analysis = customer_segments.groupBy('segment').agg(
    count('*').alias('customer_count'),
    avg('frequency').alias('avg_frequency'),
    avg('monetary').alias('avg_lifetime_value'),
    avg('avg_order_value').alias('avg_order_value'),
    avg('unique_products').alias('avg_unique_products')
).orderBy('segment')

print("\n=== Segment Analysis ===")
segment_analysis.show()
```

```
uilding Customer Segmentation Model (K-Means Clustering)...
```

5. Machine Learning Component - Customer Segmentation

• K-Means clustering has been used with four segments of customers.

Architecture of Pipelines for repeatable machine learning workflows; Standard Scaler for feature normalization

• The development of segment profiles

• Cluster 0: Heavy low-value buyers

• Segment 1: Regular medium-value customers

• Section 2: High-value VIP customers

• Section 3: Diverse Consumers

```
print("\n" + "="*70)
print("STRATEGIC INSIGHTS FOR SALES DEPARTMENT")
print("="*70 + "\n")

# Label the segments based on their characteristics
print("### Customer Segment Profiles:\n")
print("Segment 0: LOW-VALUE FREQUENT - Small orders, frequent purchases")
print("  → Strategy: Upsell bundles, increase avg order value\n")

print("Segment 1: MEDIUM-VALUE REGULAR - Moderate spending, good frequency")
print("  → Strategy: Loyalty programs, personalized recommendations\n")

print("Segment 2: HIGH-VALUE VIP - Large orders, highest lifetime value")
print("  → Strategy: Premium service, exclusive offers, retention focus\n")

print("Segment 3: DIVERSE SHOPPERS - High product variety")
print("  → Strategy: Cross-category promotions, new product launches\n")

# Product performance
top_products = df_sales_enriched.groupBy('product_id', 'category').agg(
    sum('total_amount').alias('revenue'),
    sum('quantity').alias('units_sold')
).orderBy(desc('revenue')).limit(5)
```

```python
# Product performance
top_products = df_sales_enriched.groupBy('product_id', 'category').agg(
    sum('total_amount').alias('revenue'),
    sum('quantity').alias('units_sold')
).orderBy(desc('revenue')).limit(5)

print("\n### Top 5 Products by Revenue:")
top_products.show(truncate=False)

# Anomaly detection - high value transactions
from pyspark.sql.functions import mean, stddev
stats = df_sales_enriched.select(
    mean('total_amount').alias('mean'),
    stddev('total_amount').alias('std')
).collect()[0]

threshold = stats['mean'] + 3 * stats['std']
anomalies = df_sales_enriched.filter(col('total_amount') > threshold)

print(f"\n### Anomaly Detection (Unusually High Transactions):")
print(f"Threshold: ${threshold:.2f}")
print(f"Anomalous transactions found: {anomalies.count()}")
anomalies.select('transaction_id', 'customer_id', 'total_amount', 'category').show(5)
```

6. Strategic Business Insights

• Product Strategy: Optimise inventory based on top-selling products by revenue

• Special treatment to VIP Client: it is necessary to provide special treatment like fast delivery, extra discount to our VIP Client.

7. Limitations & Critical Reflection

• Basic unsupervised learning may be enhanced for churn prediction by supervising the models.

Performance tuning is needed at production scale-partitioning, caching, broadcast joins.

• Batch-oriented pipeline; no streaming capability demonstrated

Databricks & Documentation Use:

The Databricks notebook entitled "Sales Pipeline - Advanced Analytics ML" documents all work, including:

• Cells of code executed and results shown

• Markdown cells describing each section

• Sample data displays and schema verification

• Segment Analysis and Model Training Results

```
print("\n" + "="*70)
print("PIPELINE SUMMARY & KEY TAKEAWAYS")
print("="*70)

print("\n1. DATA ACQUISITION")
print("   - Generated 10,000 synthetic sales transactions")
print("   - Alternative strategies: Parquet files, Kafka streams, JDBC connections")
print("   - Justification: SQL-based generation for Databricks compatibility")

print("\n2. DATA TRANSFORMATION")
print("   - Added temporal features (year, month, quarter, week)")
print("   - Calculated customer RFM metrics (Recency, Frequency, Monetary)")
print("   - Feature engineering for ML model input")

print("\n3. SPARK DATAFRAMES & SQL")
print("   - DataFrame API: Type-safe, composable, ML-ready")
print("   - Spark SQL: Business-friendly, analyst-accessible")
print("   - Hybrid approach: Best of both paradigms")

print("\n4. MACHINE LEARNING")
print("   - K-Means clustering for customer segmentation")
print("   - 4 distinct customer segments identified")
print("   - StandardScaler for feature normalization")
print("   - Pipeline for reproducible ML workflows")
```

```
print("\n5. BUSINESS IMPACT")
print("   ✓ Identify high-value customers for retention")
print("   ✓ Tailor marketing strategies by segment")
print("   ✓ Optimize inventory based on product performance")
print("   ✓ Detect unusual patterns for fraud prevention")

print("\n6. LIMITATIONS & FUTURE WORK")
print("   - Synthetic data lacks real-world complexity")
print("   - No temporal trends (seasonality, campaigns)")
print("   - Missing external factors (marketing spend, competition)")
print("   - Simple clustering; could enhance with supervised learning")
print("   - Performance tuning needed for production scale")

print("\n" + "="*70)
print("✓ PIPELINE COMPLETE - Ready for production deployment")
print("="*70 + "\n")
```

CONCLUSION:

To conclude, the project usage of big data technologies like Spark and Databricks for complex analytics, data pipelines to make business decisions. The strategy helps to bring the important data at a strategic level.

References

Academic & Technical Sources

• A. S. F. (2024). Apache Software Foundation. Spark

• Meng, X. et al. (2016) - This is the journal article for MLlib

• White, T. (2015) - Hadoop: The DefPySpark & Implementation

• Python Software Foundation (2024). - Python Documentation};

• VanderPlas, J. (2023) - Python Data Science